Blazor Conf 2026

Videogames e Collaborative Apps: è possibile con Blazor?

Nicola Paro

Un grazie agli sponsor e alle community

studio{MIT}

hunext SOFTWARE

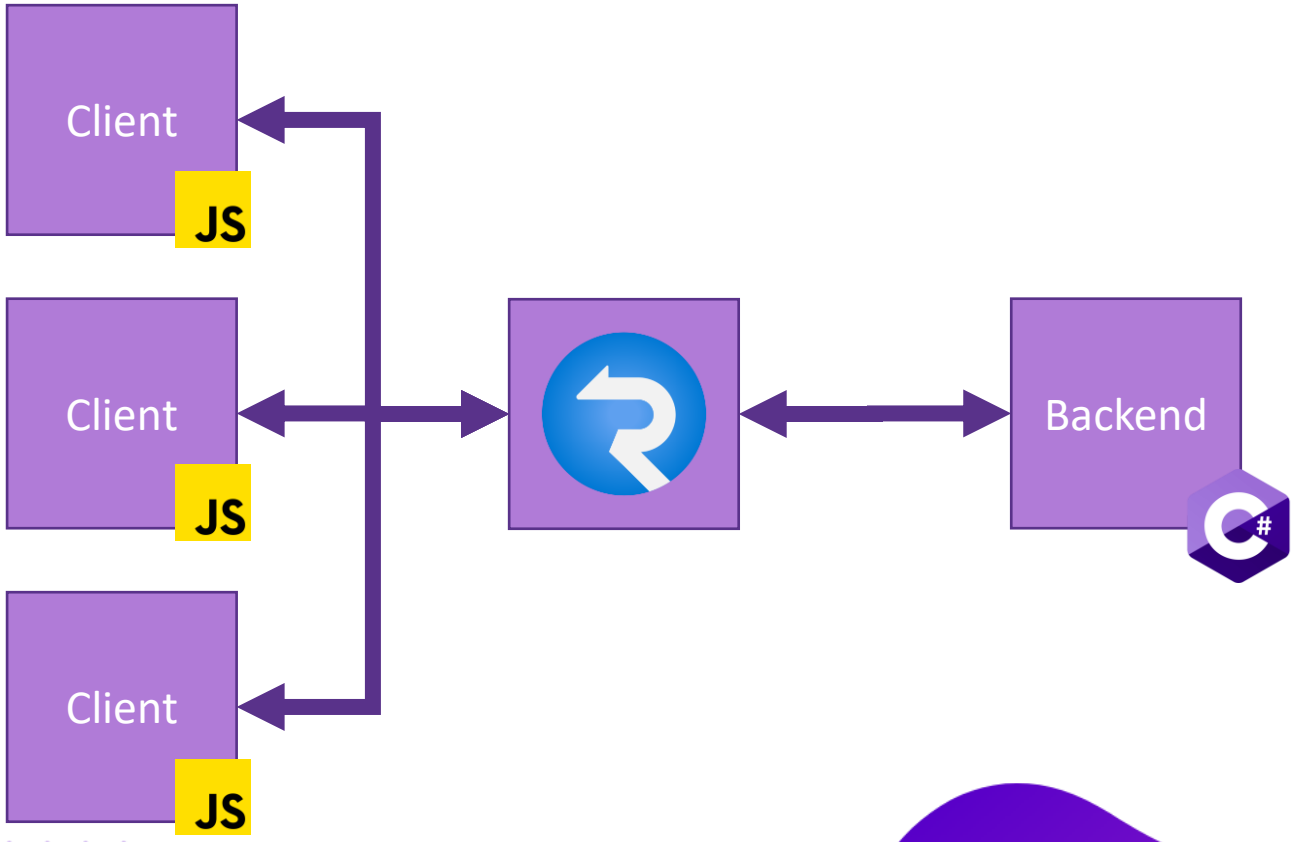
bpb

Applicazioni Collaborative

Applicazioni che permettono a più utenti di lavorare insieme in tempo reale. Condivisione immediata di dati, stato e modifiche

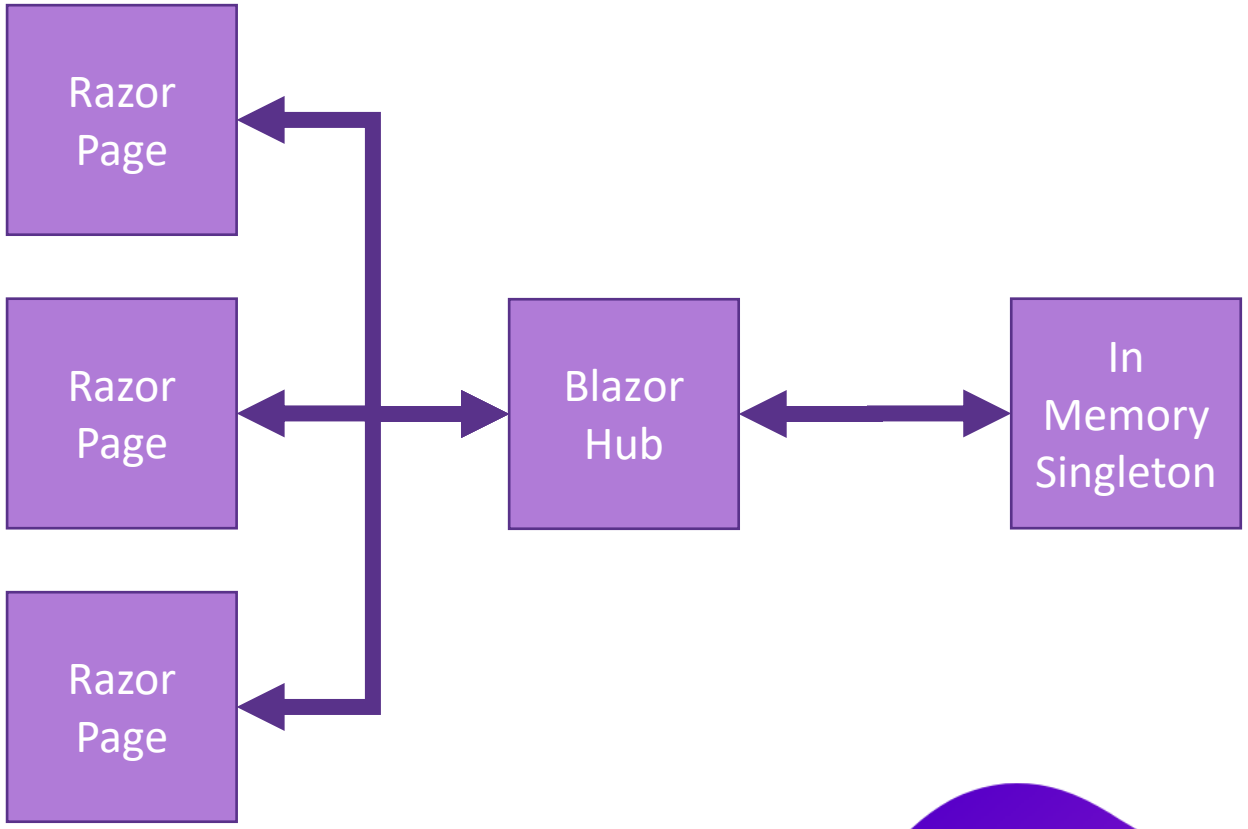
Applicazioni Collaborative

Architettura «standard»

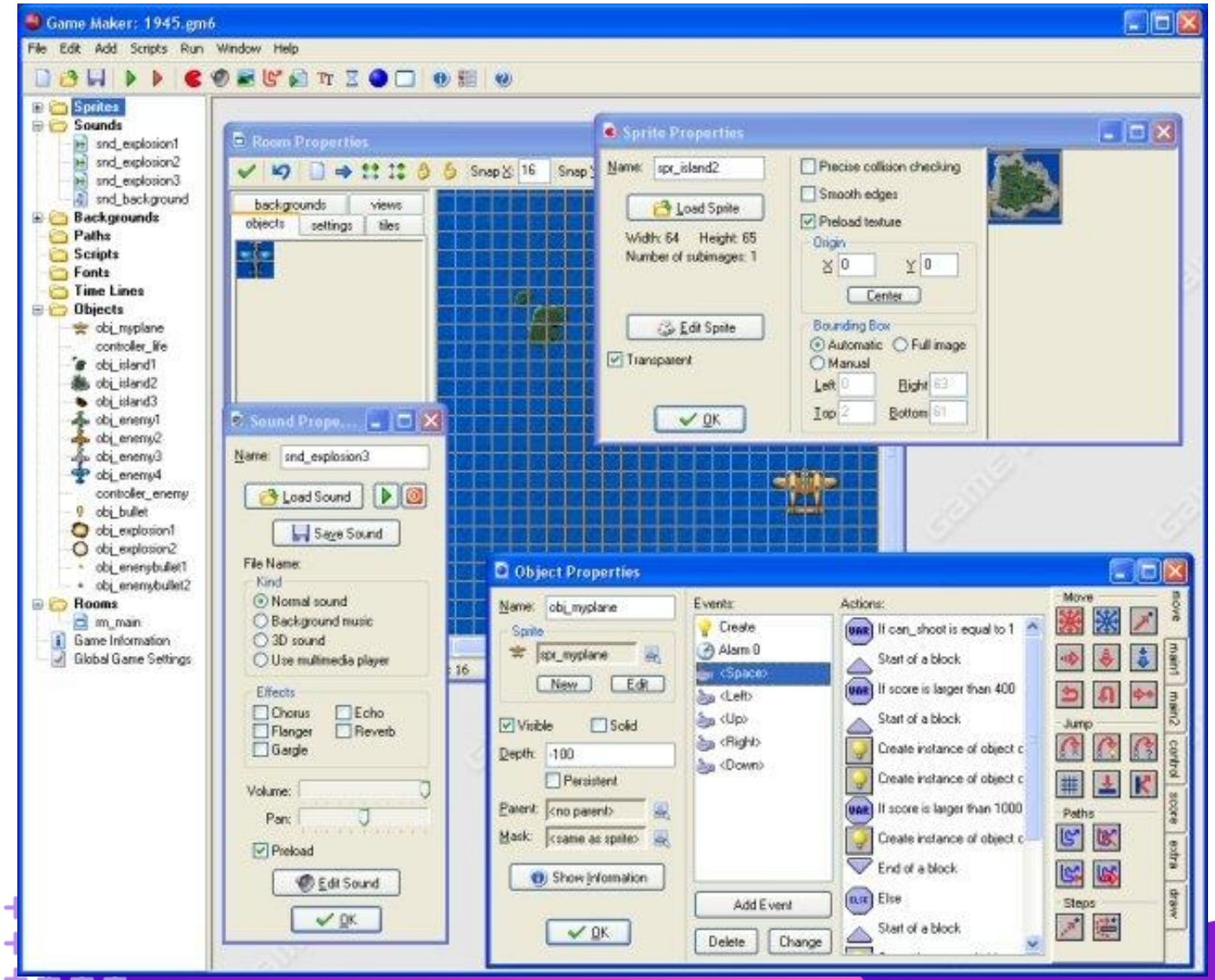


Applicazioni Collaborative

Architettura con Blazor Interactive Server



Però, tanto tempo fa…



Era molto che non creavo un videogioco…

- C’è modo di unire la mia passione per la creazione di videogiochi con le competenze che ho acquisito in questi anni?
- No, JavaScript non è strettamente una delle mie competenze
- Blazor Interactive Server può essere la strada per la creazione di un videogioco collaborativo?

Perché usare Blazor?

- Cross platform: Mi basta avere un browser per accedere.
- Non mi serve installarlo perché è accessibile via web. Posso pure aggiornarlo quando mi pare.
- Se proprio proprio mi serve creare un’app nativa, posso sempre distribuirlo come applicazione ibrida (Blazor Hybrid)
- Scrivo in C# nel web 🙂

Prima di qualsiasi riga di codice: Assets

- E’ un qualunque elemento utilizzato per costruire un videogioco.
- È tutto ciò che il gioco “usa” per mostrare, suonare o far accadere qualcosa.

Immagini: sprite, tileset, icone, sfondi.

Audio: musiche, effetti sonori, voci.

Modelli 3D e animazioni.

Testi: dialoghi, interfacce, menu.

Dati: mappe di livello, configurazioni, script.

Assets > Immagini > Tilesets

Un tileset è una raccolta di piccole tessere grafiche (i tile), tutte della stessa dimensione, utilizzate per costruire gli ambienti del gioco (pavimenti, pareti, piattaforme, prati, ecc.).

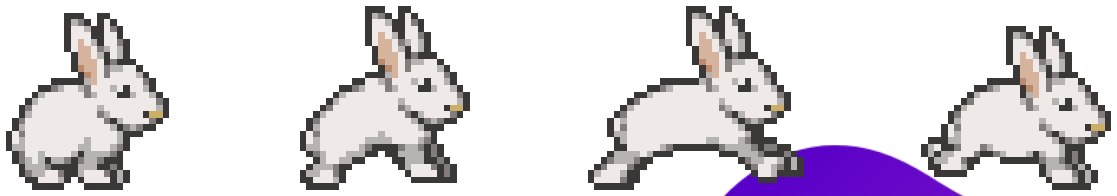
- Ogni tile è un elemento quadrato o rettangolare (per esempio 16×16 o 32×32 pixel).
- Il livello viene costruito componendo questi elementi come pezzi di un mosaico.
- Permettono di creare scenari complessi con un numero ridotto di immagini, ottimizzando spazio e performance.

Sono i “mattoni” che compongono il mondo di gioco.

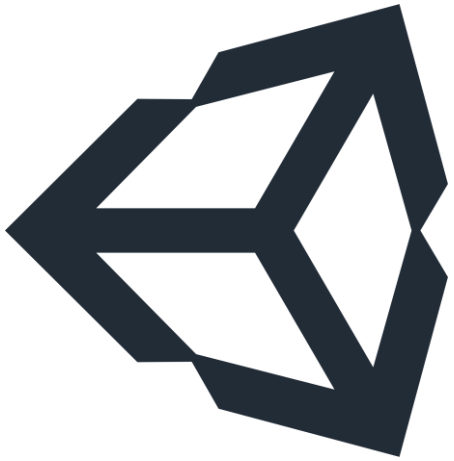
Assets > Immagini > Sprites

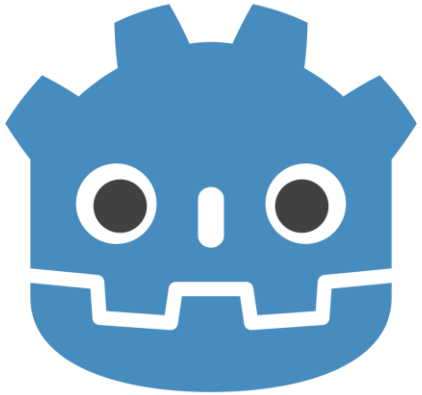
Gli sprite sono immagini 2D utilizzate per rappresentare personaggi, oggetti, nemici, proiettili o qualunque elemento “mobile” all’interno del gioco.

- Sono elementi indipendenti che vengono disegnati sopra lo sfondo.
- Possono includere animazioni, costituite da più fotogrammi (ad esempio camminata, salto, attacco).
- Vengono gestiti dal motore di gioco per permettere movimenti, collisioni e cambi di stato.



Che Game Engine Scegliamo?





Non esiste un Game Engine per Blazor

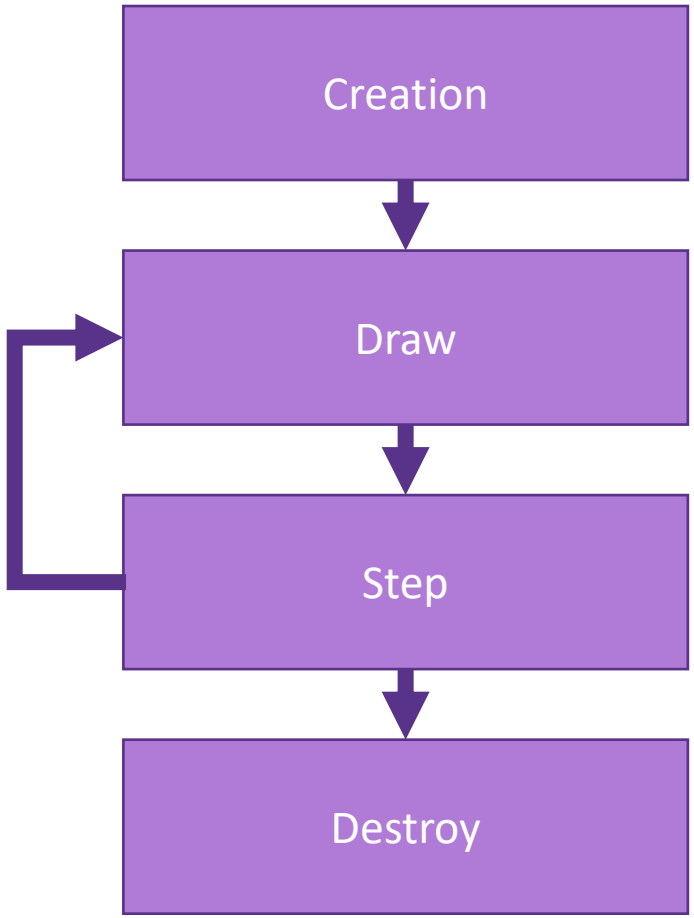
Costruiamo Il nostro Game Engine

Game
la classe base per il gioco, gestisce il game loop e lo stato globale

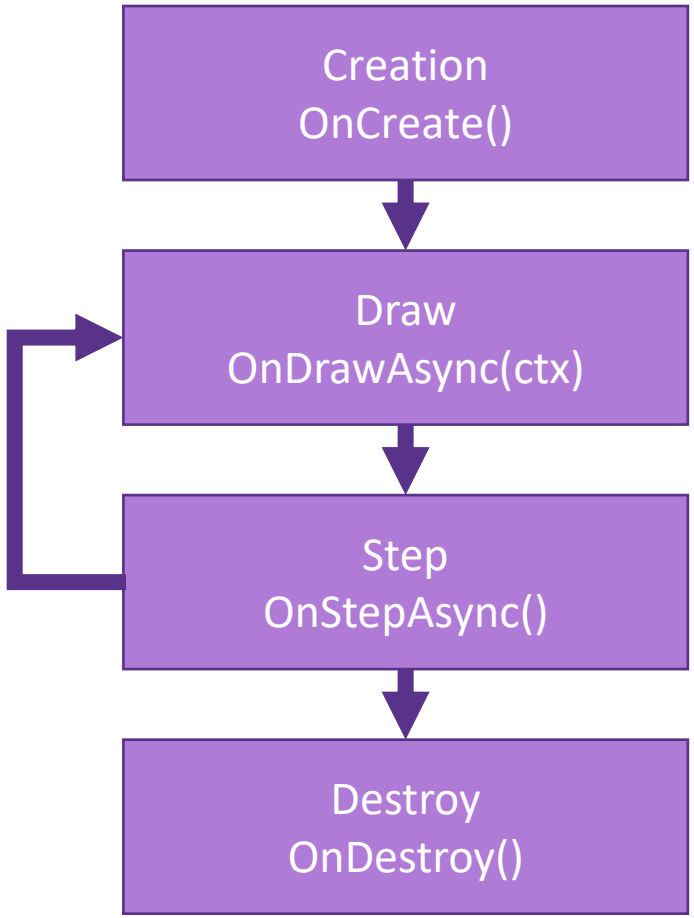
GameObject
entità del gioco che possono essere istanziate, aggiornate e renderizzate

GameView
il componente Razor che collega il game loop al rendering Canvas

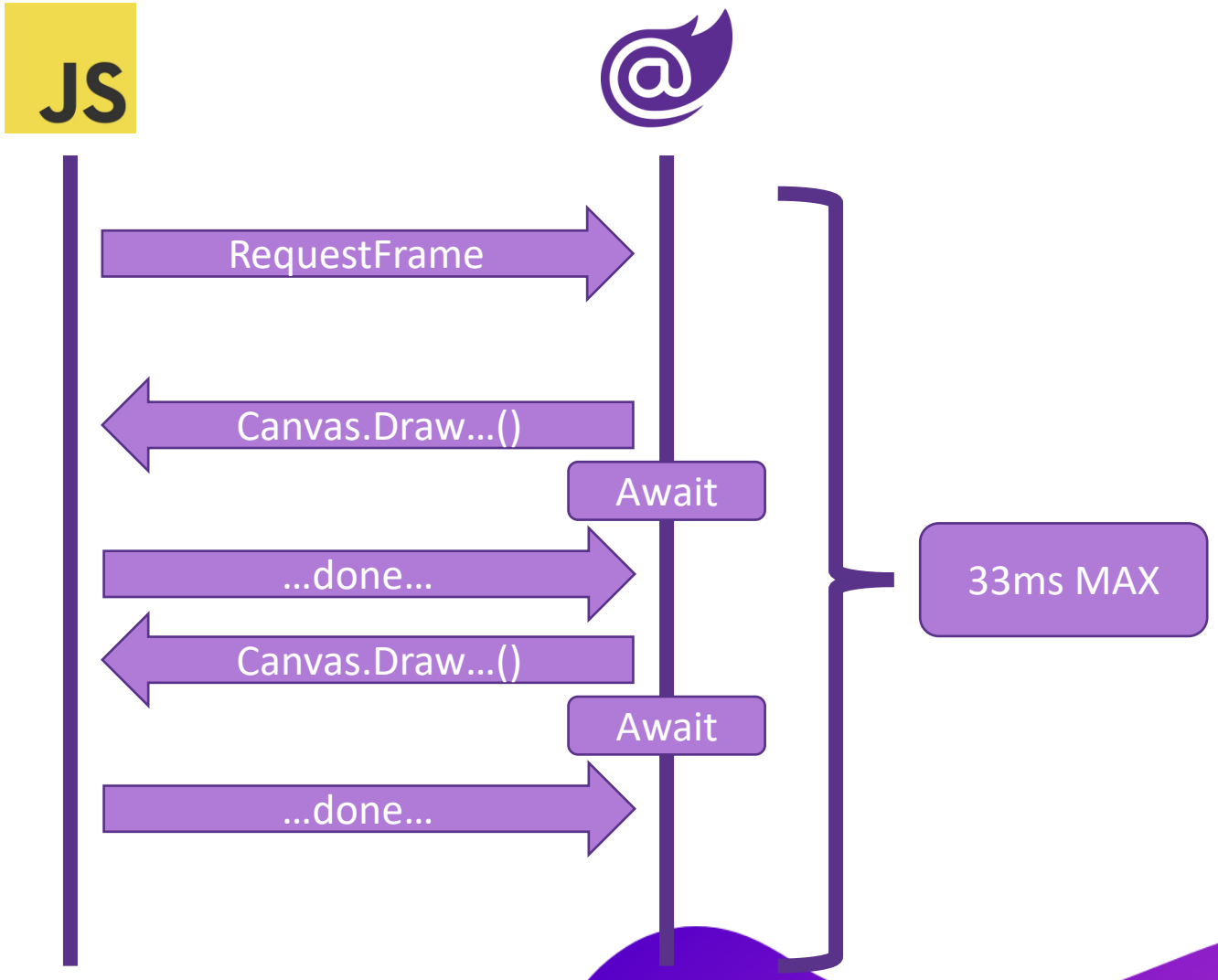
Game Engine: Game Entity Lifecycle



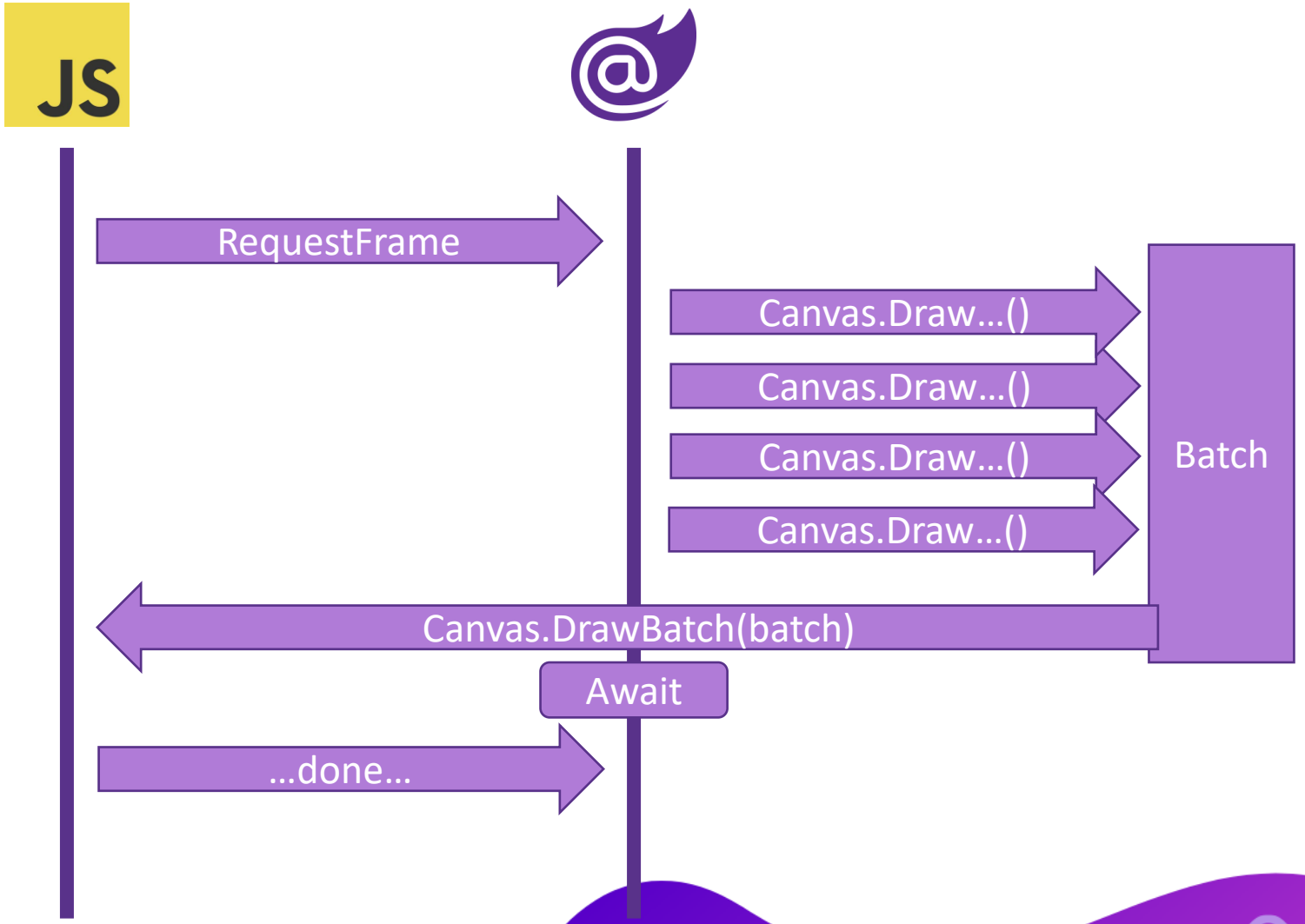
Game Engine: Game Entity Lifecycle



Game Engine: Game Entity Lifecycle



Game Engine: Game Entity Lifecycle



Canvas Library for Blazor

https://github.com/BlazorExtensions/Canvas

In your index.html file (WebAssembly Apps) or _Host.cshtml (Server Apps) file, place a reference to the library's script file:

```
<script src="_content/Blazor.Extensions.Canvas/blazor.extensions.canvas.js">
</script>
```

In your _Imports.razor add the following using entry:

```
@using Blazor.Extensions.Canvas
```

In the component where you want to place a canvas element, add a BECanvas. Make sure to set the ref to a field on your component:

```
<BECanvas Width="300" Height="400" @ref="_canvasReference" ></BECanvas>
```

Canvas Library for Blazor

```
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    var context = await canvasReference.CreateCanvas2DAsync();

    await context.BeginBatchAsync();

    await context.SetFillStyleAsync("green");
    await context.FillRectAsync(10, 100, 100, 100);

    await context.SetFontAsync("48px serif");
    await context.StrokeTextAsync("Hello Blazor!!!", 10, 100);

    await context.EndBatchAsync();
}
```

Game Loop – JS Interop

Il game loop è gestito da JavaScript per garantire un timing preciso con requestAnimationFrame:

```
export async function executeFrame(dotnetReference, callbackFunctionName) {
    await dotnetReference.invokeMethodAsync(callbackFunctionName);
    setTimeout(() => {
        window.requestAnimationFrame(() => executeFrame(dotnetReference, callbackFunctionName));
    }, 1000 / (window.fps));
};



export function setFPS(fps) {
    window.fps = fps;
}
```

- requestAnimationFrame sincronizza con il refresh del browser
- Il setTimeout permette di limitare gli FPS (es. 60 FPS)
- La chiamata asincrona a C# (invokeMethodAsync) mantiene il loop reattivo

Game Loop – C#

Ogni frame esegue questa sequenza nel metodo ExecuteFrameAsync:

```
internal async Task ExecuteFrameAsync(RenderingContext context)
{
    // Snapshot delle istanze (per evitare problemi di concorrenza)
    var instances = this.instances.ToArray();

    // FASE DI RENDERING
    if (context is Canvas2DContext canvas2DContext)
    {
        await OnDrawAsync(canvas2DContext);           // Draw del Game
        foreach (var obj in instances)
            await obj.OnDrawAsync(canvas2DContext);   // Draw di ogni GameObject
    }

    // FASE DI UPDATE
    await OnStepAsync();                               // Step del Game
    foreach (var obj in instances)
        await obj.ExecuteStepAsync();                  // Step di ogni GameObject
}
```

- Lo snapshot ToArray() previene modifiche concorrenti durante l'iterazione
- Il rendering avviene **prima** dell'update logico

Gestione Degli Input

Il Problema: Distinguere "Hold" da "Pressed"

Un gioco deve distinguere tra:

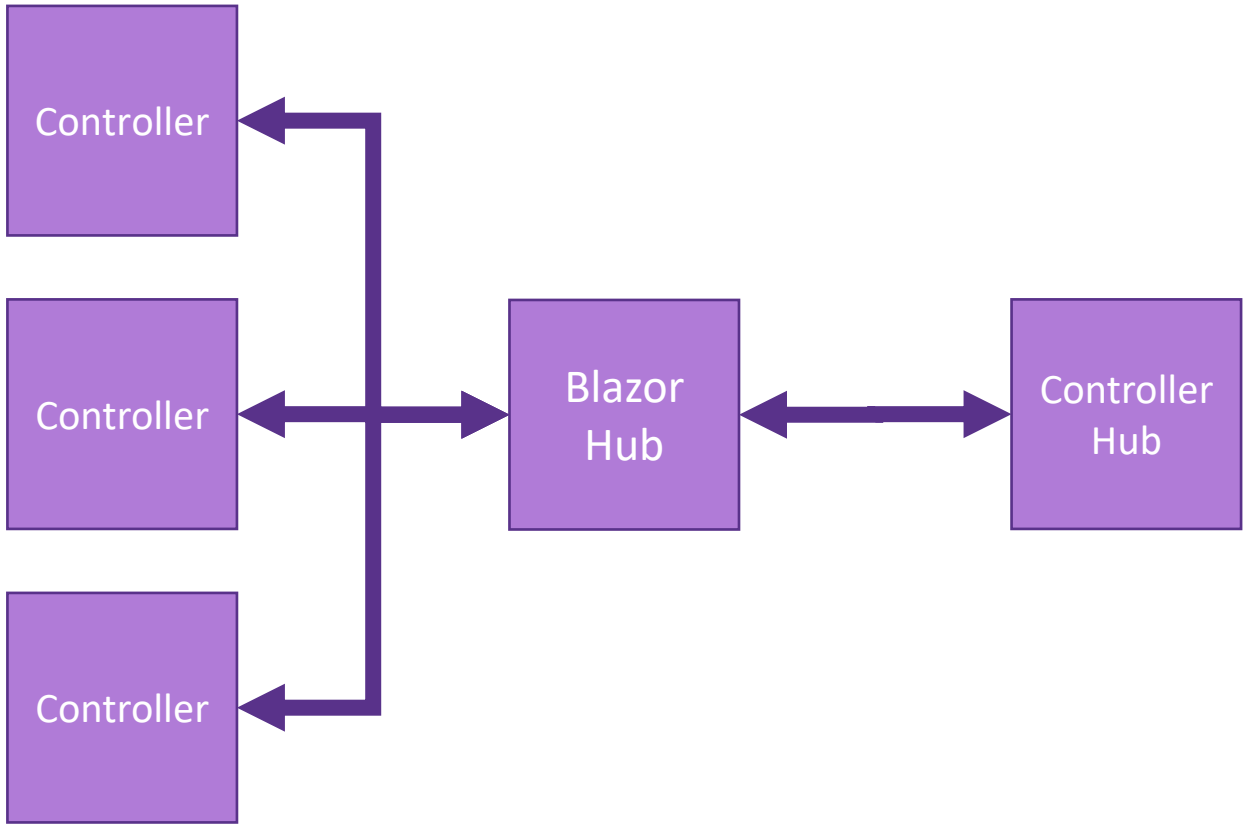
- **Check**: "Il tasto è premuto?" (tenuto premuto)
- **CheckPressed**: "Il tasto è stato appena premuto?" (solo il primo frame)
- **CheckReleased**: "Il tasto è stato appena rilasciato?"

Frame	Event	keyboardPressed["Space"]	previousKeyboardPressed["Space"]	CheckPressed	Check
1	❌ Nothing	FALSE	FALSE	FALSE	FALSE
2	⬇️ KeyDown Event	TRUE	FALSE	TRUE	TRUE
3	➖ Nothing (hold)	TRUE	TRUE	FALSE	TRUE
4	➖ Nothing (hold)	TRUE	TRUE	FALSE	TRUE
5	⬆️ KeyUp Event	FALSE	TRUE	FALSE	FALSE

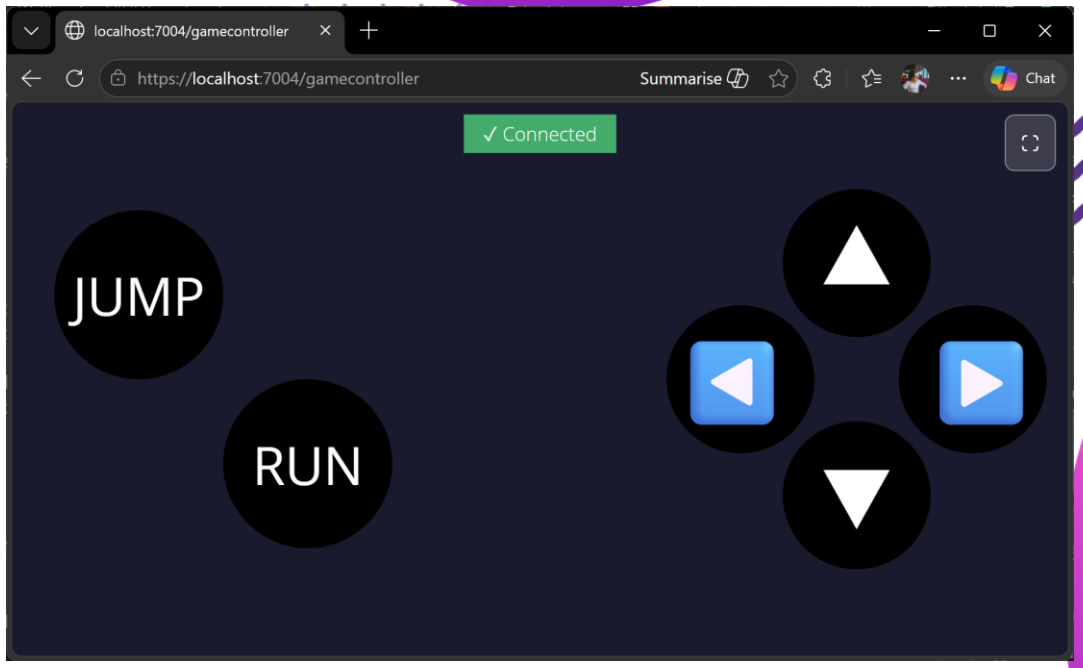
Gestione Degli Input

```
// Stato corrente e precedente
private readonly Dictionary<string, bool> keyboardPressed = new();
private readonly Dictionary<string, bool> previousKeyboardPressed = new();


// Aggiornamento ad ogni frame
foreach (var key in keyboardPressed.Keys)
    previousKeyboardPressed[key] = keyboardPressed[key];


// Logica di rilevamento
public sealed override bool KeyboardCheckPressed(string key)
{
    if (!keyboardPressed.TryGetValue(key, out bool pressed))
        return false;
    if (!previousKeyboardPressed.TryGetValue(key, out bool previousPressed))
        return false;

    // Premuto adesso ma NON nel frame precedente
    return !previousPressed && pressed;
}
```

Controller virtuali?



Ed I controller virtuali?




```
public interface IGameControllerHub
{
    event Action? OnControllersChanged;
    event Action<Guid, int>? OnVibrate;

    int ControllerCount { get; }
    void Vibrate(Guid controller, int durationMs);

    IEnumerable<Guid> GetControllers();
    void RegisterController(Guid controller);
    void UnregisterController(Guid controller);

    Task SendControllerInputUpAsync(Guid controller, string inputId);
    Task SendControllerInputDownAsync(Guid controller, string inputId);

    void Update();
    bool ControllerInputCheck(Guid controller, string inputId);
    bool ControllerInputCheckPressed(Guid controller, string inputId);
    bool ControllerInputCheckReleased(Guid controller, string inputId);
}
```

GameObject

```
public abstract class GameObject(Game game) : GameEntity(game)
{
    public SpriteAsset? Sprite { get; set; }
    public BoundingBox? BoundingBox { get; set; }


    private float imageIndex;
    public int ImageIndex { get => (int)imageIndex; set => imageIndex = value; }
    public float ImageSpeed { get; set; } = 1;


    public float OriginalX { get; init; }
    public float OriginalY { get; init; }
    public float X { get; set; }
    public float Y { get; set; }
    public float VSpeed { get; set; }
    public float HSpeed { get; set; }
    public float VAcceleration { get; set; }
    public float HAcceleration { get; set; }


    // ...
}
```

GameObject – Esempio

```
public class Player : GameObject
{
    private readonly PlayerSpriteAsset sprite;

    // Il costrutor riceve dipendenze via DI
    public Player(Game game, PlayerSpriteAsset sprite) : base(game)
    {
        this.sprite = sprite;
        Sprite = sprite;
        BoundingBox = new BoundingBox(0, 0, 16, 32);
    }

    public override void OnCreate()
    {
        // Chiamato dopo l'istanziazione
        ImageSpeed = 0.2f;
    }

    public override async ValueTask OnStepAsync()
    {
        // Logica di movimento
        if (KeyboardCheck(KeyboardKeys.ArrowRight))
            HSpeed = 2;
        else if (KeyboardCheck(KeyboardKeys.ArrowLeft))
            HSpeed = -2;
        else
            HSpeed = 0;

        VAcceleration = 0.5f; // Gravità
    }
}
```

Fisica e Collisioni

```
public bool IsCollidingWith(GameObject other)
{
    if (BoundingBox is null || other.BoundingBox is null)
        return false;


    // AABB (Axis-Aligned Bounding Box) collision
    return !(X + BoundingBox.X + BoundingBox.Width <= other.X + other.BoundingBox.X ||
             X + BoundingBox.X >= other.X + other.BoundingBox.X + other.BoundingBox.Width ||
             Y + BoundingBox.Y + BoundingBox.Height <= other.Y + other.BoundingBox.Y ||
             Y + BoundingBox.Y >= other.Y + other.BoundingBox.Y + other.BoundingBox.Height);
}
```

Due box collidono se nessuna delle seguenti condizioni è vera:

- Box1 è completamente a sinistra di Box2
- Box1 è completamente a destra di Box2
- Box1 è completamente sopra Box2
- Box1 è completamente sotto Box2

Altre Funzioni

Funzione	Descrizione
IsPointEmpty(x, y)	Verifica se la posizione alle coordinate indicate non contiene elementi solidi.
IsPointFree(x, y)	Controlla se l'oggetto può occupare la posizione indicata, testando i quattro angoli del suo bounding box.
IsCollidingWith<T>(out others)	Cerca collisioni con tutte le istanze di tipo *T* presenti nel gioco e restituisce l'elenco degli oggetti che collidono.
IsCollidingWith(other)	Verifica se i bounding box di questo oggetto e di un altro si sovrappongono.
MoveOutsideSolid(deltaX, deltaY, resolution)	Avanza nella direzione definita dallo spostamento proposto finché non trova un punto libero da solidi.
MoveOutsideSolid(direction, maxDistance, resolution)	Avanza lungo una direzione angolare fino a raggiungere il primo punto non solido.
MoveContactSolid(deltaX, deltaY, resolution)	Avanza gradualmente nella direzione specificata fino a toccare un'area solida, fermandosi immediatamente prima del contatto.
MoveContactSolid(direction, maxDistance, resolution)	Avanza lungo una direzione angolare finché non incontra un solido, fermandosi al primo punto di contatto.

Blazor Conf 2026

Collaborative Games?

DEMO

Considerazioni

- In questo setup, Blazor Server non è probabilmente la scelta più indicata per creare un videogame e renderlo accessibile al pubblico direttamente «online»

- Può avere senso utilizzare questo Game Engine con Blazor WebAssembly o Blazor Hybrid per far girare il gioco direttamente sul dispositivo dell’utente, e gestire in altro modo la comunicazione con il server.

- La soluzione con Blazor Server è fattibile solamente se anche i client connessi sono all’interno della stessa rete. La latenza del networking è il limite a questa soluzione.

- Poiché Blazor funziona «ad evento», l’impiego di questa tecnologia per applicazioni collaborative in realtime interne, con un setup infrastrutturale minimale è sicuramente valida.

Blazor Conf 2026

Q & A

Blazor Conf 2026

Grazie!

github.com/nicolaparo/BlazorConf2026Demo

Nicola Paro

Solution Architect

Microsoft® MVP Most Valuable Professional

linkedin.com/in/nicolaparo